

CSC 330 Programming Languages

Lecture #3

Grammar and Parsing

Introduction

What exactly is a programming language? This may seem like an easy question, but you would probably be surprised at the answer. Is a language the same as the editor? Is Python the same as IDLE, PyCharm or Notepad++? Obviously not. Different editors may support different languages, but a language is not equal to the editor. Ok, then, is a language equal to its interpreter? Is the language Python the same as the command “python” typed in the terminal to run a Python program? Is Java the same as “javac” or “java” typed in the terminal? Actually, no. These are compilers (or interpreters) that are used to either convert the language into binary or run the language directly. Is a language comprised of the syntax and rules of that language? Now we are getting closer. Essentially, a programming language is a set of written syntax and production rules that describe all valid “sentences” in the language. But how do we represent this? We could take a language like Java and try to write out the syntax for a function header (aka a function signature). We could write it like this:

```
RETURN-TYPE  FUNC-NAME  ( PARAM-TYPE  PARAM-NAME, ...) { ... }
```

This gets across the basic meaning and allows someone to plug in values for each spot to create something like this:

```
int multByTwo (int x) { ... }
```

But what about visibility (aka access modifier)? Let’s add that:

```
ACCESS-MOD  RETURN-TYPE  FUNC-NAME  ( PARAM-TYPE  PARAM-NAME, ...) { ... }
```

Now we can have the following:

```
public int multByTwo (int x) { ... }
```

What about the “static” keyword? Or the “final” keyword? These are optional so we need to have some way of expressing this, but we will worry about that later.

```
ACCESS-MOD  STATIC  FINAL  RETURN-TYPE  FUNC-NAME  ( PARAM-TYPE  PARAM-NAME, ...) { ... }
```

Now we can have multiple options:

```
public static int multByTwo (int x) { ... }
```

```
private final float multByTwo (float x) { ... }
```

```
protected static final double multByTwo (double x) { ... }
```

This is going to take a while. We need to have a defined syntax to represent syntax! We need to establish rules and symbols to cover the syntax of all possible features of a language. In the end, every single possible expression/statement in the language must be represented!

Grammar

Before we can go any further, we need to understand that when you write in a programming language, all of this is really just one long string. For us to describe the valid syntax of a language, we must describe how our large string is broken up. Each symbol (such as keywords, identifiers, values and operators) must be described. These symbols are more commonly called "tokens".

Let's recall our syntax for a function header:

```
ACCESS-MOD STATIC FINAL RETURN-TYPE FUNC-NAME ( PARAM-TYPE PARAM-NAME, ...) { ... }
```

What is "STATIC" and "FINAL"? In Java, this is represented by the keyword "static" and the keyword "final". How do we know the difference between this being some rule to define vs actual text (i.e. "STATIC")? Also, how do we represent that this is actually optional, i.e. we don't have to mark every function as static? We should clearly define what STATIC and FINAL are and what exact characters make up each of these things.

```
STATIC = 'static' | ε  
FINAL  = 'final' | ε
```

This denotes that the STATIC token is represented by the literal word "static" in all lowercase. Similar for the FINAL token. However, we use pipe '|' to represent multiple choices (we read it as "or"). The Greek letter Epsilon 'ε' is used to signify an empty string (think of it like "ε mpty"), meaning the STATIC term can either be the word "static" in all lowercase or it could be nothing. This makes it optional.

How about the ACCESS-MOD token? Well, that can be defined as follows:

```
ACCESS-MOD = 'public' | 'private' | 'protected' | ε
```

This means an access modifier can be either the literal text "public", "private", "protected", or you can omit it entirely.

RETURN-TYPE is next. This one has multiple choices:

```
RETURN-TYPE = 'void' | 'int' | 'float' | 'String' | 'char'
```

Note that we cannot omit this. Did we cover all types? Not really. First, a return type can be any of the simple primitives and we have left some out. Let's try to define a token for those to help us out:

```
PRIMITIVE-TYPE = 'byte' | 'short' | 'int' | 'long' | 'float' | 'double' | 'char' | 'boolean'
```

We also have our string type:

```
STRING = 'String'
```

Don't forget about the void type. It is only valid in a few places, and it isn't a primitive type. So, let's define a new token just for it:

```
VOID = 'void'
```

So now return type can be defined as:

```
RETURN-TYPE = PRIMITIVE-TYPE | STRING | VOID
```

Have we covered everything now? Nope. What about the data types we define ourselves? If we create a custom class, we have essentially created our own data type, and any method can return an object of our type. So, we will need to define that as well:

```
USER-DEFINED-TYPE = IDENTIFIER
```

But what exactly is an IDENTIFIER? An identifier is just a name for something defined, such as a class, field, or method. We will need to define the rules here:

```
LETTER = 'A'|'B'|'C'|'D'|'E'|'F'|'G'|'H'|'I'|'J'|'K'|'L'|'M'|
        |'N'|'O'|'P'|'Q'|'R'|'S'|'T'|'U'|'V'|'W'|'X'|'Y'|'Z'|
        |'a'|'b'|'c'|'d'|'e'|'f'|'g'|'h'|'i'|'j'|'k'|'l'|'m'|
        |'n'|'o'|'p'|'q'|'r'|'s'|'t'|'u'|'v'|'w'|'x'|'y'|'z'|
DIGIT = '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'|
UNDERSCORE = '_'
IDENTIFIER = UNDERSCORE IDENTIFIER-REST
            | LETTER IDENTIFIER-REST
IDENTIFIER-REST = UNDERSCORE IDENTIFIER-REST
                | LETTER IDENTIFIER-REST
                | NUMBER IDENTIFIER-REST
                | ε
```

Suddenly, things just got a whole lot more complicated! We first define a single letter, which can be uppercase A through Z or lowercase a through z. We then define a single digit, which can be 0 through 9. We define underscore as the '_' character. Then we put it all together to show that an identifier can start with either an underscore or a letter, then can be followed by an underscore, a letter, or a number.

Now we can complete our RETURN-TYPE rule:

```
RETURN-TYPE = PRIMITIVE-TYPE | STRING | VOID | USER-DEFINED-TYPE
```

Recall our function header definition:

```
ACCESS-MOD STATIC FINAL RETURN-TYPE FUNC-NAME ( PARAM-TYPE PARAM-NAME, ...) { ... }
```

Our function name is just an identifier, so we can replace it with:

```
ACCESS-MOD STATIC FINAL RETURN-TYPE IDENTIFIER ( PARAM-TYPE PARAM-NAME, ...) { ... }
```

Instead of putting "(" and ")" in our rule and confusing it with a grouping mechanism, let's define these as actual tokens, to show they are literal text. Let's also define the comma token.

```
LPAREN = '('
RPAREN = ')'
COMMA = ','
```

Now we have:

```
ACCESS_MOD STATIC FINAL RETURN_TYPE IDENTIFIER LPAREN PARAM-TYPE PARAM-NAME COMMA ... RPAREN { ... }
```

We can't really just put an ellipsis ... in the rule and expect everyone to know what we mean. Let's define a list of parameters more formally:

```
FUNC-PARAMS = PARAM-TYPE PARAM-NAME FUNC-PARAMS-REST  
FUNC-PARAMS-REST = COMMA PARAM-TYPE PARAM-NAME FUNC-PARAMS-REST  
                  |  $\epsilon$ 
```

To be fair, a parameter name is really just an identifier, and its type is just a data type, so we can rewrite this as follows:

```
FUNC-PARAMS = DATA-TYPE IDENTIFIER FUNC-PARAMS-REST  
FUNC-PARAMS-REST = COMMA DATA-TYPE IDENTIFIER FUNC-PARAMS-REST  
                  |  $\epsilon$ 
```

We can also define the function body as follows:

```
LBRACE = '{'  
RBRACE = '}'  
BODY = STATEMENT BODY  
      | EXPRESSION BODY  
      |  $\epsilon$ 
```

So, now the full function becomes:

```
ACCESS_MOD STATIC FINAL RETURN_TYPE IDENTIFIER LPAREN FUNC-PARAMS RPAREN LBRACE BODY RBRACE
```

Let's group together some parts of this to make it easier to read:

```
FUNC-MODS = ACCESS_MOD STATIC FINAL  
FUNC-HEADER = FUNC-MODS RETURN-TYPE IDENTIFIER LPAREN FUNC-PARAMS RPAREN  
FUNCTION = FUNC-HEADER LBRACE BODY RBRACE
```

Let's collect everything together and clean it up a little:

```

// keywords
ACCESS-MOD = 'public' | 'private' | 'protected' | ε
STATIC = 'static' | ε
FINAL = 'final' | ε

// types
PRIMITIVE-TYPE = 'byte' | 'short' | 'int' | 'long' | 'float' | 'double' | 'char' | 'boolean'
STRING = 'String'
VOID = 'void'

// operators and symbols
LETTER = 'A'|'B'|'C'|'D'|'E'|'F'|'G'|'H'|'I'|'J'|'K'|'L'|'M'|
        |'N'|'O'|'P'|'Q'|'R'|'S'|'T'|'U'|'V'|'W'|'X'|'Y'|'Z'|
        |'a'|'b'|'c'|'d'|'e'|'f'|'g'|'h'|'i'|'j'|'k'|'l'|'m'|
        |'n'|'o'|'p'|'q'|'r'|'s'|'t'|'u'|'v'|'w'|'x'|'y'|'z'|

DIGIT = '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'
UNDERSCORE = '_'
LPAREN = '('
RPAREN = ')'
COMMA = ','
LBRACE = '{'
RBRACE = '}'

// rules
IDENTIFIER = UNDERSCORE IDENTIFIER-REST
            | LETTER IDENTIFIER-REST

IDENTIFIER-REST = UNDERSCORE IDENTIFIER-REST
                 | LETTER IDENTIFIER-REST
                 | NUMBER IDENTIFIER-REST
                 | ε

USER-DEFINED-TYPE = IDENTIFIER
DATA-TYPE = PRIMITIVE-TYPE | STRING | VOID | USER-DEFINED-TYPE

FUNC-RETURN-TYPE = DATA-TYPE | VOID
FUNC-MODS = ACCESS-MOD STATIC FINAL
FUNC-PARAMS = DATA-TYPE IDENTIFIER FUNC-PARAMS-REST
FUNC-PARAMS-REST = COMMA DATA-TYPE IDENTIFIER FUNC-PARAMS-REST
                  | ε
FUNC-HEADER = FUNC-MODS FUNC-RETURN-TYPE IDENTIFIER LPAREN FUNC-PARAMS RPAREN
BODY = STATEMENT BODY
      | EXPRESSION BODY
      | ε

FUNCTION = FUNC-HEADER LBRACE BODY RBRACE

```

By now you should be able to see how these grammars can become quite large! This is all needed just to describe a function. There are plenty more things to cover in a programming language. Of course, we can reuse things we have already defined, such as identifiers, letters, primitive_types, and so on. Check out the grammar for C# <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/grammar>

This is what a language is! A language's grammar defines exactly what the language is and every valid "sentence" in the language can be derived from the grammar.

Mistake

We actually have a mistake in our grammar. There is something that is perfectly fine, even common in the Java language that our grammar does not support. Can you spot the mistake? Take a moment to look over the grammar at home and see if you can find the problem.

Context Free Grammar

What would happen if we had the following code:

```
int int (String void) { }
```

Would this be recognized by our grammar? The characters 'i', 'n', 't' do match an identifier so we could technically call it an IDENTIFIER in our grammar, couldn't we? Actually, our grammar would not allow this! When writing our rules, order matters. Since PRIMITIVE-TYPE, STRING, and VOID are written before IDENTIFIER, this means that words like "int" and "void" will NOT be recognized as identifiers and therefore would not be allowed to be used in the way we have used them above. Had we put IDENTIFIER first in our grammar, then we would have had a problem. So issues like this are easy to solve.

What about this code:

```
foo bar (int i) { }
```

Is this allowed? Well it depends on how "foo" is defined:

```
class foo { } // if foo is defined like this, then the line of code is ok
float foo; // if foo is defined like this, then the line of code is NOT ok
```

Whether the code above is ok or not depends on the context of how "foo" is defined. The problem is that grammars are often "context free". This means that we really have no way of specifying context. To fix this problem, we will have to carry around context when we write our interpreter. This context is commonly called a "symbol table". More on this later.

Notation

The grammar above is written in a way to make it easier to understand, but this isn't a supported notation. The main notation for expressing grammars is BNF. BNF stands for Backus-Naur Form and is very similar to the grammar above but with slightly different notation.

BNF uses <> to denote what we call "terminals" and "non-terminals". A non-terminal is a symbol that is composed of other non-terminals (and terminals). It can be broken down into smaller parts. In contrast a terminal is a token that is composed solely of characters. For example, a terminal might be:

```
STATIC = 'static'
```

Here the terminal token STATIC is composed of the literal text "static". A non-terminal however is composed of other non-terminals and possibly terminals, such as:

```
DATA-TYPE = PRIMITIVE-TYPE | STRING | VOID | USER-DEFINED-TYPE
```

Normally we put terminal tokens and their definitions into the lexer and non-terminals into the parser. But we will cover that distinction later.

Let's rewrite our DATA-TYPE non-terminal in proper BNF form:

```
<DATA-TYPE> ::= <PRIMITIVE-TYPE> | <STRING> | <VOID> | <USER-DEFINED-TYPE>
```

We actually don't need to put things in ALL-CAPS either. Because of the <> notation, there is already a distinction between tokens and literal text, so putting these in lowercase is perfectly fine. We can also represent ranges with three dots ... this means our <letter> and <digit> tokens become a lot simpler to write. We use semicolons (instead of //) to denote comments in our grammar. Here is the same grammar as before, but in proper BNF form:

```
; keywords
<access-mod> ::= 'public' | 'private' | 'protected' | ε
<static>      ::= 'static' | ε
<final>       ::= 'final'  | ε

; types
<primitive-type> ::= 'byte' | 'short' | 'int' | 'long' | 'float' | 'double'
                  | 'char' | 'boolean'
<string> ::= 'String'
<void>   ::= 'void'

; operators and symbols
<letter> ::= 'A'...'Z' | 'a'...'z'
<digit>  ::= '0'...'9'
<underscore> ::= '_'
<lparen> ::= '('
<rparen> ::= ')'
<comma>  ::= ','
<lbrace> ::= '{'
<rbrace> ::= '}'

; rules
<identifier> ::= <underscore> <identifier-rest>
               | <letter> <identifier-rest>
<identifier-rest> ::= <underscore> <identifier-rest>
                    | <letter> <identifier-rest>
                    | <number> <identifier-rest>
                    | ε
<user-defined-type> ::= <identifier>
<data-type> ::= <primitive-type>
               | <string>
               | <void>
               | <user-defined-type>

<func-return-type> ::= <data-type> | <void>
<func-mods> ::= <access-mod> <static> <final>
<func-params> ::= <data-type> <identifier> <func-params-rest>
<func-params-rest> ::= <comma> <data-type> <identifier> <func-params-rest>
                    | ε
<func-header> ::= <func-mods> <func-return-type> <identifier> <lparen>
                 <func-params> <rparen>
<body> ::= <statement> <body>
          | <expression> <body>
          | ε
<function> ::= <func-header> <lbrace> <body> <rbrace>
```

Lexical Analysis

How does this grammar get put into a compiler or interpreter? When you see all these rules, how do you build software to recognize it? Let's look at a simple example.

Let's say your source code has the following line of code:

```
void foo(int i) { }
```

The first step to understanding this code is to convert it into terminal tokens. This process is called Lexical Analysis. After this process runs, the following stream of tokens would be produced:

```
<void> <identifier> <lparen> <primitive-type> <identifier> <rparen> <lbrace> <rbrace>
```

To explain how this actually works behind the scenes, we need to focus on the terminal rules in our grammar. The ones we will focus on for this code snippet will be:

```
<void> ::= 'void'
<primitive-type> ::= 'byte' | 'short' | 'int' | 'long' | 'float' | 'double'
                  | 'char' | 'boolean'
<lparen> ::= '('
<rparen> ::= ')'
<lbrace> ::= '{'
<rbrace> ::= '}'
```

We also need to examine this rule:

```
<identifier> ::= <underscore> <identifier-rest>
              | <letter> <identifier-rest>
<identifier-rest> ::= <underscore> <identifier-rest>
                   | <letter> <identifier-rest>
                   | <number> <identifier-rest>
                   | ε
```

For now, we can consider this to be a terminal token since it really just breaks down into the matched characters. Later on, we will see how we can rewrite this to make it look more like a true terminal token.

To tokenize an input stream, we need to examine these rules a little closer. We need to build what's called a deterministic finite automaton (or DFA for short). Essentially this will help us consume the input, character-by-character and determine which tokens it converts into. We will take a "greedy" approach, meaning we will grab as many characters as we can to make a single token.

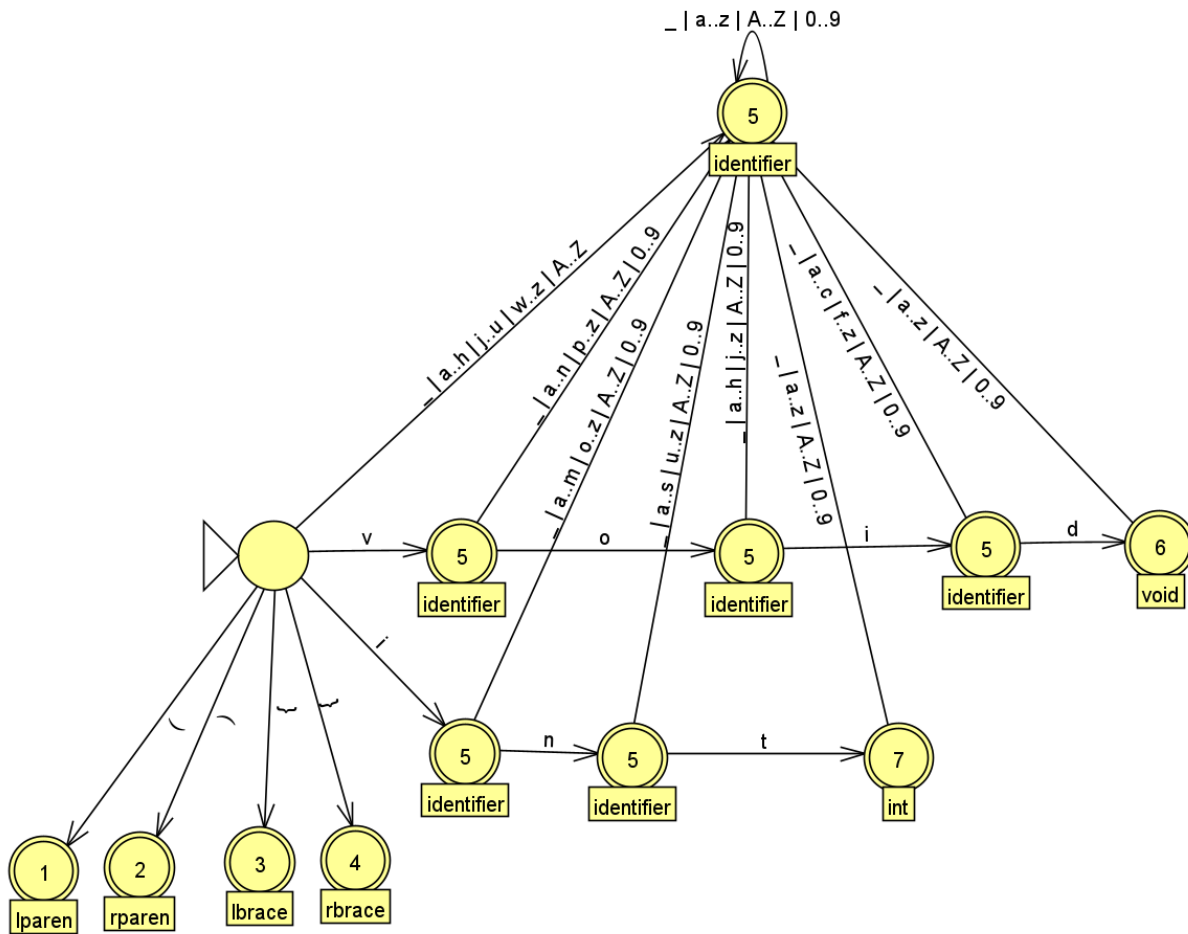
To get an intuition of how this works, consider the input string from above:

```
void foo(int i) { }
```

The code will consume 'v' and see what all this matches. It matches <void> and <identifier>. We don't know which one it is yet, so we consume the next character. Now we have 'vo'. So far it still could either be an identifier or the void keyword. We then grab the 'i' then the 'd'. So now we have 'void'. We know this is now the 'void' keyword. However, perhaps the next character is another letter? If so, this would be an identifier. Since we are taking a greedy approach, we want to get the longest character sequence that matches. The next character is a space, which doesn't match <void> nor <identifier>, so we stop grabbing characters. We have

successfully matched the <void> terminal token. We can throw this into our token stream and grab the next set of characters.

A DFA helps us do this exact process. We build the DFA from our terminal rules and send the input stream through it. For this set of terminal rules, this is what a possible DFA would be:



For the sake of brevity, I've omitted all types in the `<primitive-type>` token except for 'int'. You start at the node with the triangle pointing to it. When you consume a letter from the input, you move to that node. As you can see, consuming a 'v' moves you to another node. The nodes that form a completed token are circled twice. These are "terminal nodes". The number in these nodes indicates a numbering system for our tokens. Normally when writing a parser, you give each token a number. In our DFA, `<lparen>` is 1, `<rparen>` is 2, `<lbrace>` is 3, and so on according to the number in that node. As soon as we consume the 'd' in 'void', we move to the 'void' node, which is terminal. When you arrive at a terminal node, you don't immediately stop there. Instead, you keep grabbing input as long as there exists a transition to another node. When we see the space character, this doesn't match any transition and so we stop at the 'void' node. Basically, we go as far as we can in this DFA, hoping to different nodes, until there doesn't exist any transitions from the current node we are on (based on the input character we just consumed). Once that occurs, if we are on a terminal node, then we have successfully matched a token and we can add this token to our token stream. If we aren't on a terminal node, then we can conclude there must be a syntax error in the input.

This DFA is built from the terminal rules and is used for the entire input file. This allows the lexer to convert the high level code input into a token stream.

Parsing

After we have our token stream, we need to match it to our non-terminal rules, also known as our "production rules". Later on, we will learn about tools that allow you to inject custom code every time a production rule is matched. This is how we can create our parser. We can evaluate our expressions as they are matched. Let's see how to parse these tokens manually first.

We need to build a parse tree from this. A parse tree will show the breakdown of production rules and how they are derived from our tokens. In a compiler, we would traverse this parse tree to generate lower level code, such as assembly or byte code. In an interpreter, we can evaluate these expressions directly. This is essentially a turning point between compilers and interpreters, as up until this point their work is largely the same.

Let's remind ourselves of the production rules we will need for this token stream:

```
// rules
<data-type> ::= <primitive-type> | <string> | <void> | <identifier>

<func-return-type> ::= <data-type> | <void>
<func-mods> ::= <access-mod> <static> <final>
<func-params> ::= <data-type> <identifier> <func-params-rest>
<func-params-rest> ::= <comma> <data-type> <identifier> <func-params-rest>
                        | ε
<func-header> ::= <func-mods> <func-return-type> <identifier> <lparen>
                  <func-params> <rparen>
<body> ::= <statement> <body>
          | <expression> <body>
          | ε
<function> ::= <func-header> <lbrace> <body> <rbrace>
```

For this to work, we need to have a root to our parse tree. We need a starting rule, so let's add that:

```
<unit> ::= <function>
```

This shows us that the first thing we should look for is a function. Obviously in a real language, this starting rule would look something more like this:

```
<unit> ::= <line-of-code> ';' <unit>
          | ε
```

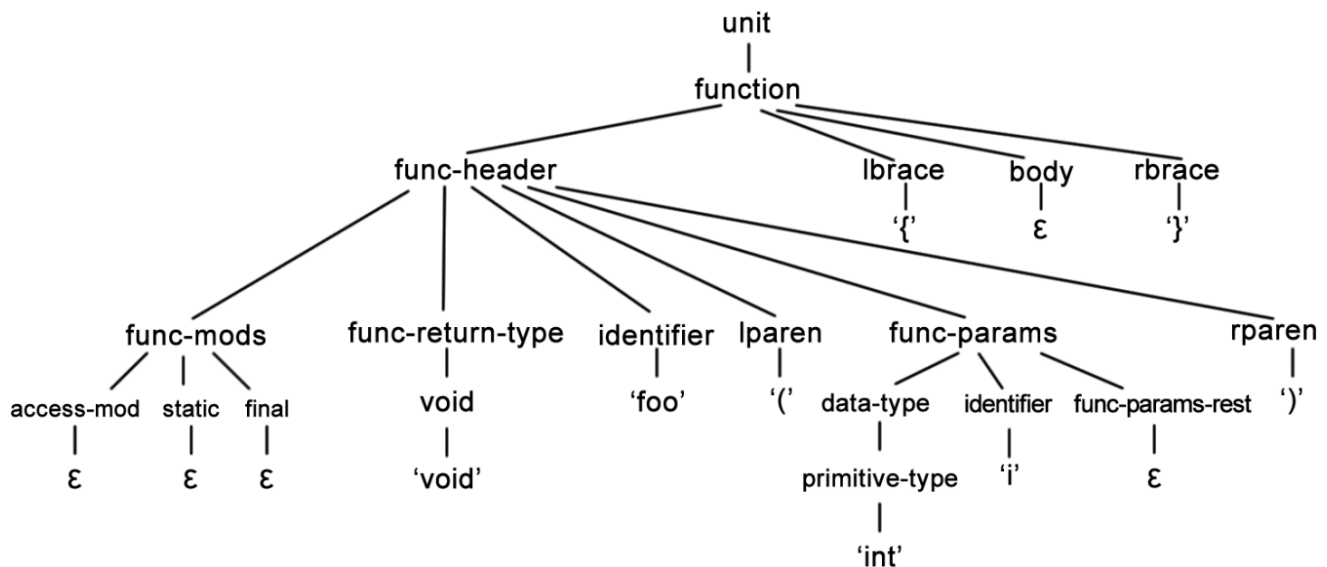
This shows that a compilation unit or just "unit" is composed of 0 or more lines of code with semi colons at the end of each. However, for our example, having an entire compilation unit just be a function definition will be fine, as our grammar really only focuses on that.

As a reminder, here is our token stream that we got from the lexer:

```
<void> <identifier> <lparen> <primitive-type> <identifier> <rparen> <lbrace> <rbrace>
```

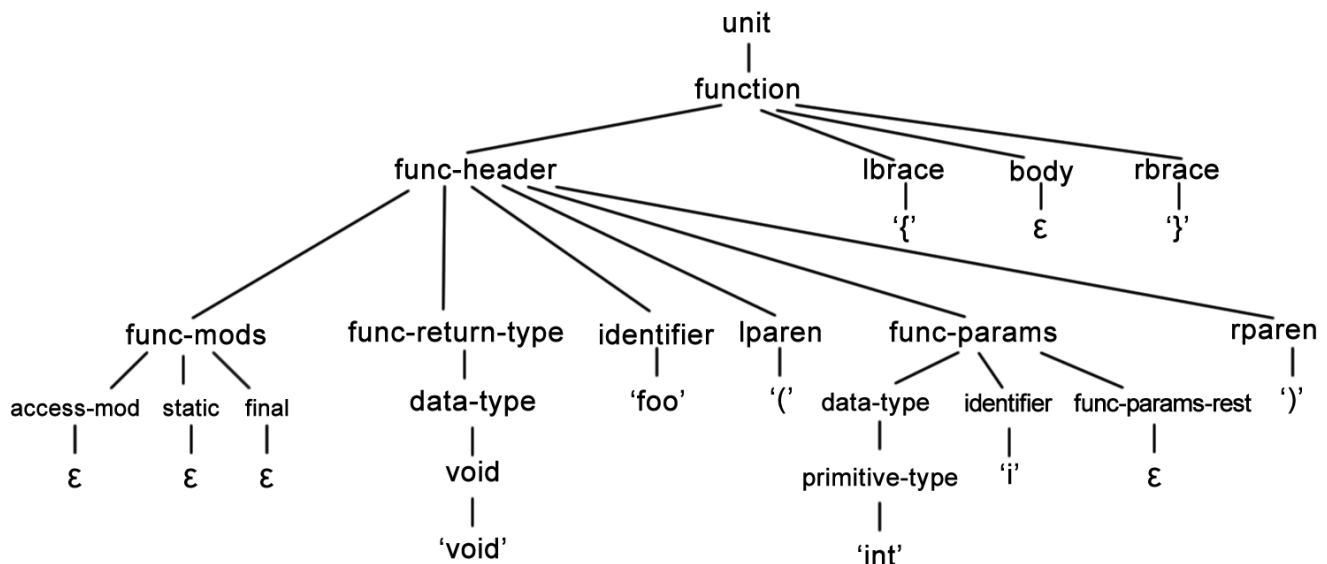
We start with our <unit> non-terminal. The only rule for a <unit> is that it breaks down into a <function>, so we make the substitution. Now we are looking to resolve a <function>. This also only has a single rule, so we break down a <function> into its parts: <func-header> <lbrace> <body> <rbrace>. We continue to do this and each time we break down a non-terminal, we make the pieces into children for that non-terminal node.

Sometimes there are multiple ways to break down a non-terminal, i.e. multiple rules. For example, a <func-return-type> could be either a <data-type> or a <void>. If we were implementing this in a compiler or parser, we would use a concept known as "look-ahead" to resolve this. Basically, we would look to the next token (without consuming it) to see which rule we should follow. Once we know which rule to take, we can then break down the non-terminal using that rule and continue to consume the tokens. Once we get down to our terminal tokens, we can stop the tree there. I went ahead and added the text inside of each terminal token just for clarity. Here is the full parse tree from our token stream:



Technically there is an ambiguity to our grammar. An ambiguous grammar is defined as having two or more possible parse trees for a given token stream. For every token stream, there should be exactly one parse tree that can be created. If no parse tree can be created, then the input is invalid and there is a syntax error. If multiple parse trees can be created, then the grammar itself is ambiguous. Ambiguous grammars can lead to different compilers generating different low level code, which can lead to our code behaving differently depending on the compiler we used. This is obviously not ideal.

The ambiguity we have with our grammar resides in the <func-return-type> rule. This could either be a <data-type> or a <void>. However, a <data-type> could also be <void>. So, if we see the token <void>, do we use <data-type> or not? Here is another valid parse tree we could get from our token stream:



Fortunately, the resolution to this is straightforward. For our simple grammar, we can just remove the <void> token from our <data-type> non-terminal:

```
<data-type> ::= <primitive-type> | <string> | <identifier>
```

This is fine since 'void' really isn't a true data type anyway. Its use is restricted to certain places, so it makes more sense to just put <void> directly in those places it can go, instead of all places that a data type is expected. This change to our grammar makes the top parse tree (the one without <data-type>) the only valid one.

Some languages keep their ambiguous grammars, like C. The creators of the language rely upon extra rules they created that accompanies the language and states how to resolve these issues. Still some undetermined behavior exists in the C language and different compilers will handle it in different ways. Normally its best to resolve the ambiguity at the grammar level, unless this causes the grammar to explode in size as possibly new non-terminals may need to be created along with new production rules.

Associativity and Precedence

You may have noticed that some of the rules has recursion in them. The rule seems to have itself as a non-terminal. This is by design and shows that we can have more than one occurrence. As an example:

```
<body> ::= <statement> <body>
        | <expression> <body>
        | ε
```

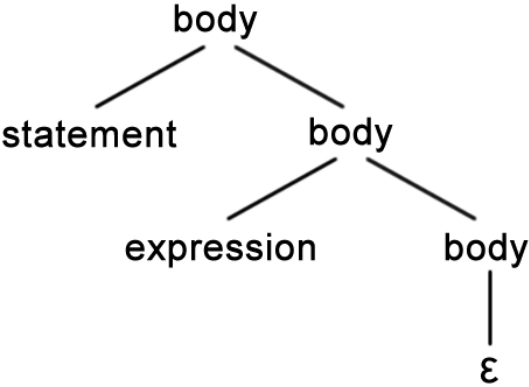
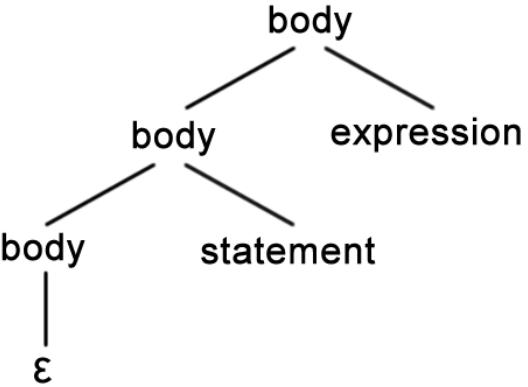
This shows that the body can be composed of a mix of several statements and expressions. But does placement of the recursive part matter? Would the following rule be the same:

```
<body> ::= <body> <statement>
        | <body> <expression>
        | ε
```

No, it won't be, since this would produce a different parse tree. Given the token stream:

<statement> <expression>

Here are the parse trees for the two rules:

	
<pre><body> ::= <statement> <body> <expression> <body> ε</pre>	<pre><body> ::= <body> <statement> <body> <expression> ε</pre>

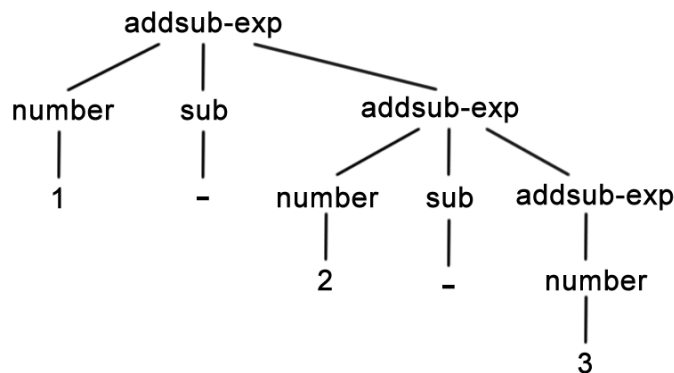
Since we have different parse trees, this is technically a different grammar. But does it really matter in this case? When we are evaluating the pieces and interpreting the code, the order of the evaluation happens from left to right in a parse tree. This is similar to doing an inorder traversal. So, for the tree on the left, we will evaluate the statement, then the expression, then the empty body. For the tree on the right, we will evaluate the empty body, then the statement, then the expression. So, it doesn't really matter much in this case. However, what about if we have math expressions? Consider the following grammar:

```
<addsub-exp> ::= <number> <add> <addsub-exp>
                | <number> <sub> <addsub-exp>
                | <number>
```

If given the following input:

1 - 2 - 3 which would be tokenized into: <number> <sub> <number> <sub> <number>

The parse tree would look like:

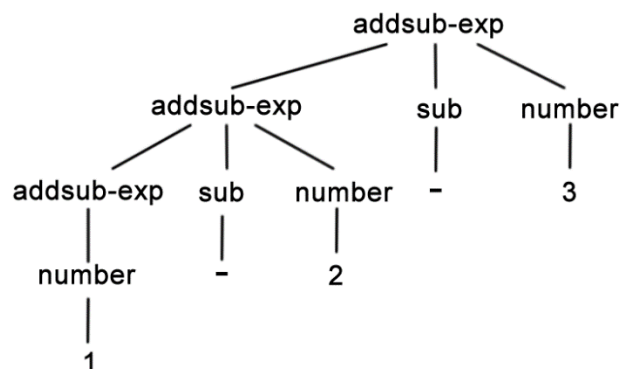


This would result in an evaluation of $1 - (2 - 3)$ which would be +2

On the other hand, if we apply the recursion on the left instead of the right, we will get:

```
<addsub-exp> ::= <addsub-exp> <add> <number>
                | <addsub-exp> <sub> <number>
                | <number>
```

Which would result in this parse tree:



And an evaluation of: $(1 - 2) - 3$ which is -4

We can see an important trend forming. It seems as though the choice of a left recursive or right recursive rule decides the order of evaluation. We call this the associativity of an operation. An operation can be left associative (which is done by making the rule left recursive) or it can be right associative (which is done by making the rule right recursive). This determines whether chained expressions are evaluated from right to left or left to right.

Obviously for most math operators we want left associativity, meaning we want left recursive rules so the expression can be evaluated from left to right:

$((\text{number op number}) \text{ op number}) \text{ op number}) \dots$

One of the exceptions to this, is exponentiation. That must be evaluated from right to left. So, if you see the following math:

$2 ** 3 ** 4 ** 5$ i.e. $2^{3^{4^5}}$

You would evaluate $4 ** 5$ first to get 1,024. You would then evaluate $3 ** 1024$ and then 2 raised to that result. So, the power operator needs to be right associative. A rule for this could be:

```
<math-exp> ::= <math-exp> <add> <number>
              | <math-exp> <sub> <number>
              | <math-exp> <mult> <number>
              | <math-exp> <div> <number>
              | <number> <power> <math-exp>
              | <number>
```

Now we can have more complex input and still get the associativity correct:

$1 - 2 - 3 * 4 ** 2 ** 1$

What's that? You say your calculator is giving you a different result? Did we miss something? Well, aside from this grammar being ambiguous, it has no concept of precedence. We know that power comes first, then mult/div, then add/sub. This grammar puts all operators at the same level. To fix that, we need to chop this up into multiple non-terminals. An operators precedence (i.e. the order in which it is evaluated when other operators are present) is determined by its depth in the parse tree. The more non-terminals we have to break down, the further an operator gets in the tree. Remember that the tree is evaluated on the way up, so leaves are going first, and the root expression will be last. To introduce precedence, we need to have more non-terminals:

```
<math-exp> ::= <addsub-exp>
```

```
<addsub-exp> ::= <addsub-exp> <add> <multdiv-exp>
                | <addsub-exp> <sub> <multdiv-exp>
                | <multdiv-exp>
```

```
<multdiv-exp> ::= <multdiv-exp> <mult> <power-exp>
                | <multdiv-exp> <div> <power-exp>
                | <power-exp>
```

```
<power-exp> ::= <number> <power> <power-exp>
                | <number>
```

$$1 - 2 - 3 * 4 ** 2 ** 1$$

```

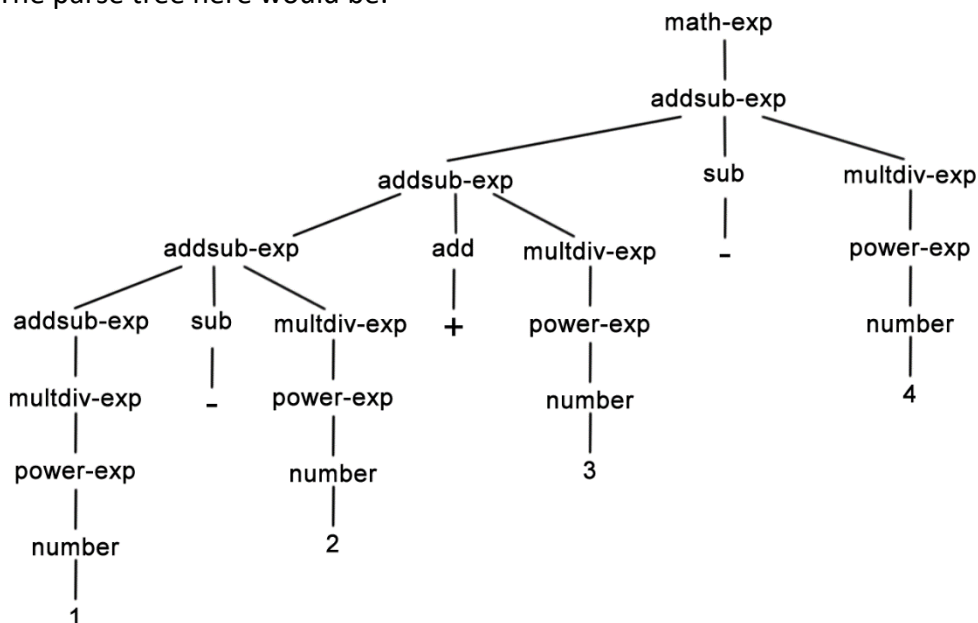
graph TD
    math-exp --> addsub-exp1[addsub-exp]
    addsub-exp1 --> addsub-exp2[addsub-exp]
    addsub-exp1 --> sub1[sub]
    addsub-exp1 --> multdiv-exp1[multdiv-exp]
    addsub-exp2 --> addsub-exp3[addsub-exp]
    addsub-exp2 --> sub2[sub]
    addsub-exp2 --> multdiv-exp2[multdiv-exp]
    addsub-exp3 --> multdiv-exp3[multdiv-exp]
    addsub-exp3 --> sub3[sub]
    addsub-exp3 --> multdiv-exp4[multdiv-exp]
    addsub-exp3 --> power-exp1[power-exp]
    multdiv-exp3 --> power-exp2[power-exp]
    multdiv-exp3 --> number1[number]
    power-exp2 --> number2[number]
    number1 --> 1[1]
    sub2 --> minus1[-]
    multdiv-exp2 --> power-exp3[power-exp]
    multdiv-exp2 --> number2[number]
    power-exp3 --> number3[number]
    number2 --> 2[2]
    multdiv-exp1 --> multdiv-exp5[multdiv-exp]
    multdiv-exp1 --> mult[mult]
    multdiv-exp1 --> power-exp4[power-exp]
    multdiv-exp5 --> power-exp5[power-exp]
    multdiv-exp5 --> number3[number]
    power-exp5 --> number4[number]
    number3 --> 3[3]
    mult --> asterisk[*]
    power-exp4 --> number4[number]
    number4 --> 4[4]
    power --> double_star[**]
    power-exp4 --> power5[power]
    power5 --> double_star2[**]
    power-exp4 --> power-exp6[power-exp]
    power-exp6 --> number5[number]
    number5 --> 2[2]
    power5 --> power6[power]
    power6 --> double_star3[**]
    power-exp6 --> power-exp7[power-exp]
    power-exp7 --> number6[number]
    number6 --> 1[1]

```

$$(1 - 2) - (3 * (4 ** (2 ** 1)))$$

What about if we use operators at the same precedence level. Let's look at the following expression:

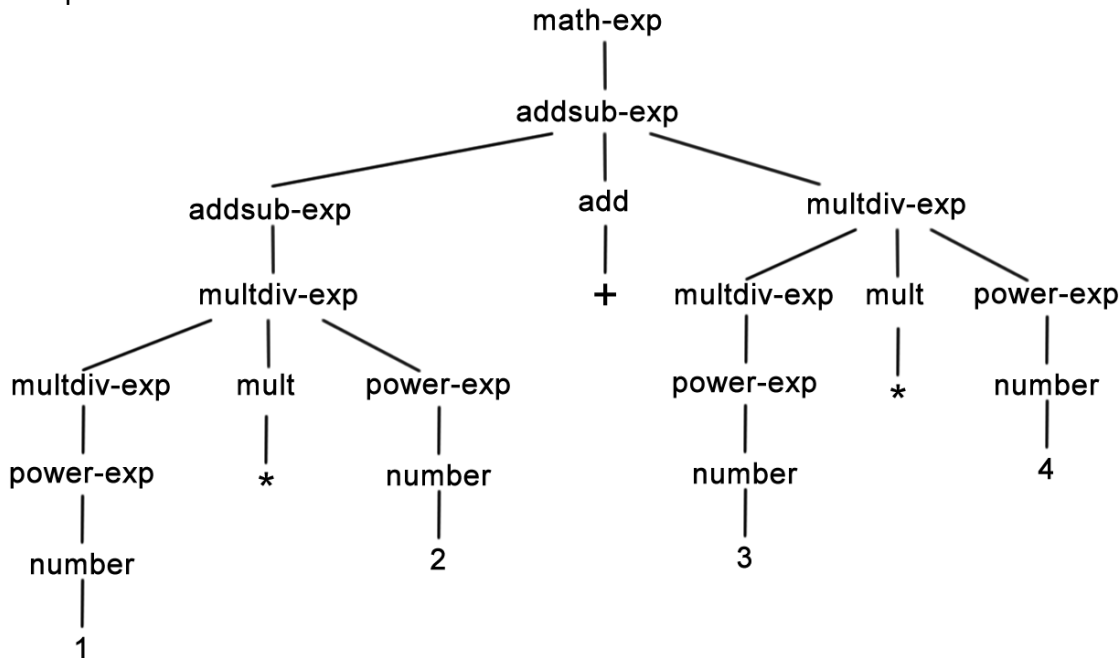
The parse tree here would be:



Let's try an expression that alternates operators of different precedence levels:

1 * 2 + 3 * 4

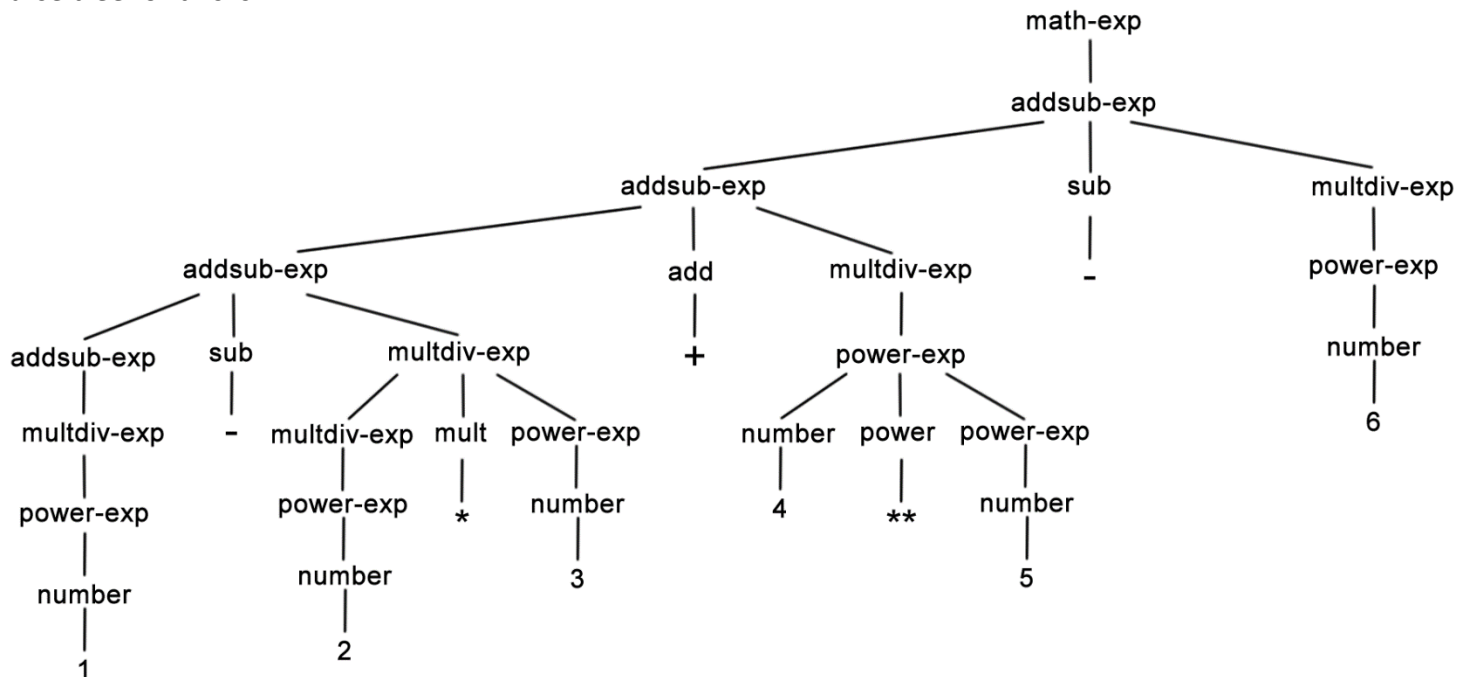
The parse tree would be:



One final expression just to make sure I exhaust all possibilities. What if the expression has a good mix of operators from different precedence levels:

1 - 2 * 3 + 4 ** 5 - 6

Parse tree for this is:

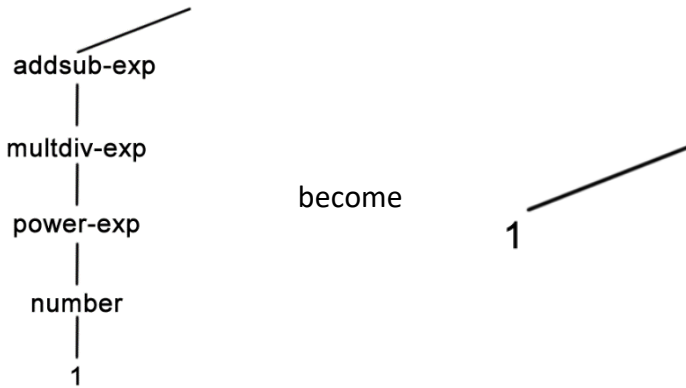


Hopefully by now you should be able to create these trees manually if given a grammar and some input to tokenize. To do this programmatically would require a stack and the operations of "shift" and "reduce". I'll leave the details of that process out of these notes, but feel free to look this up online if you are interested.

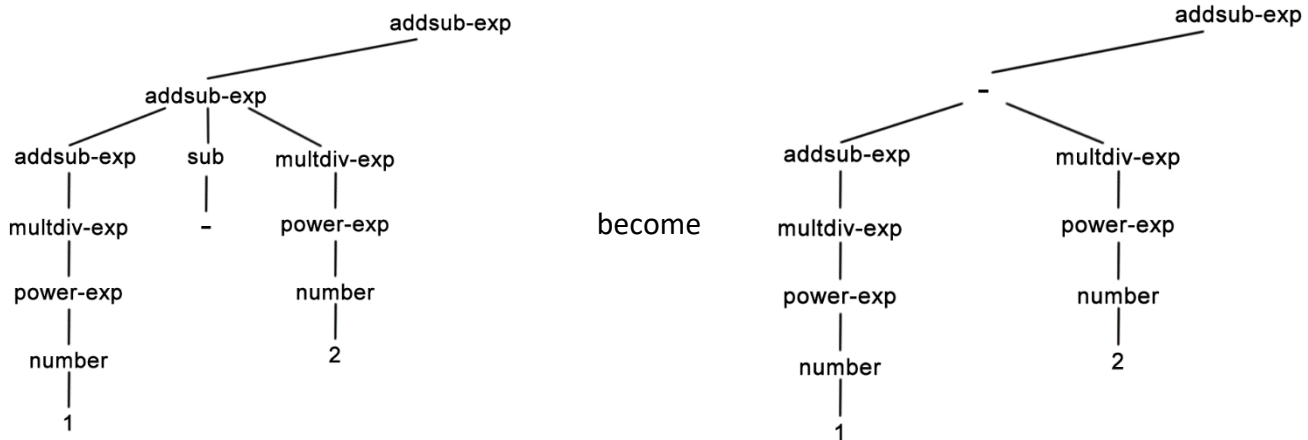
Abstract Syntax Tree

If we look at the parse tree from our grammar and input, we realize there is a lot of fluff there. Do we really need all that for showing the evaluation order? Probably not. A parse tree is overly verbose on purpose since it is showing the exact parse of the token stream to a production rule. It has to include everything. However, we can simplify this some by convert this into an "abstract syntax tree".

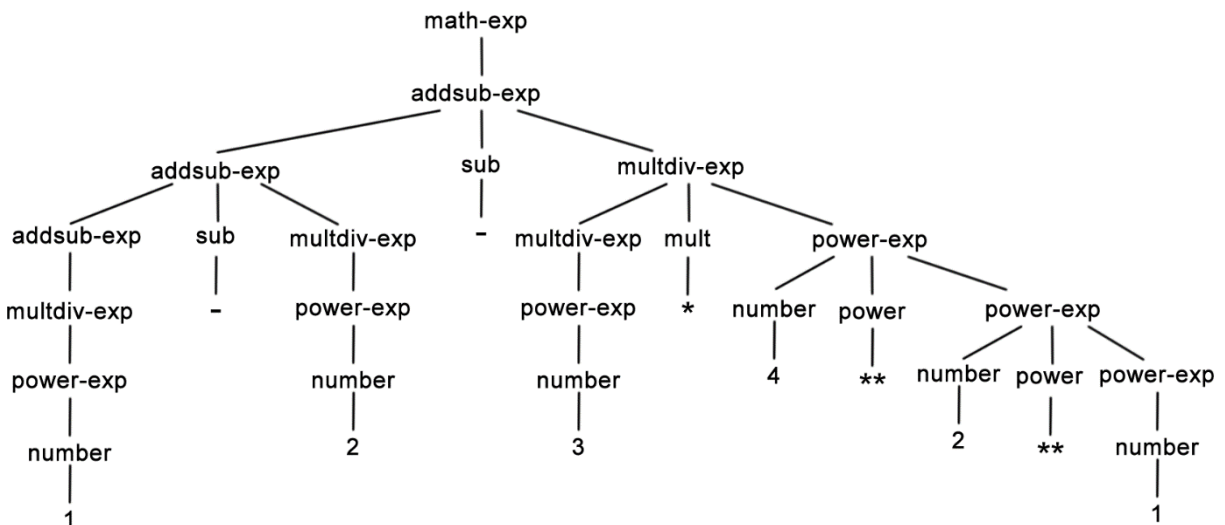
An abstract syntax tree shows us just what we need to see in order to get a sense of what is going on in the expression. It omits any paths in the tree that continually contain only a single child node. So, parts like this:



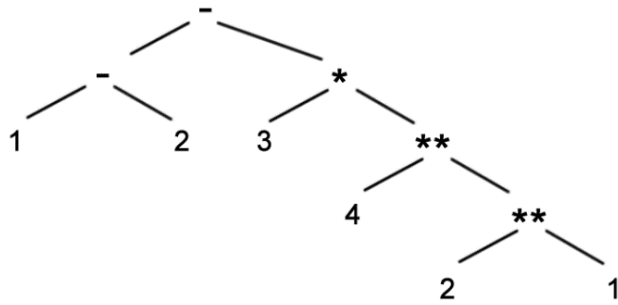
We also move the operator to be the subroot of that part of the tree. This means that parts like:



Putting this all together, we can reduce one of the parse trees from before:

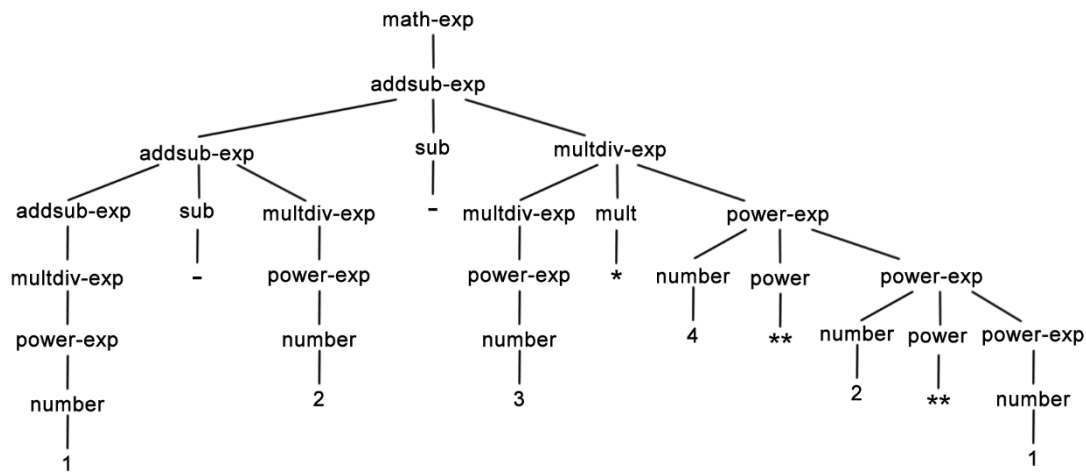


Into this abstract syntax tree:

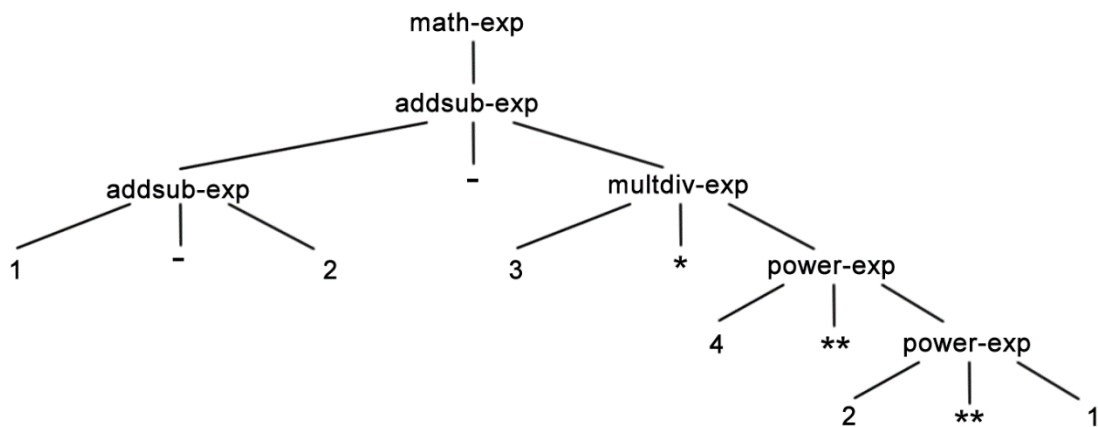


Just in case that jump from one tree to the other was too much, here are the intermediate steps:

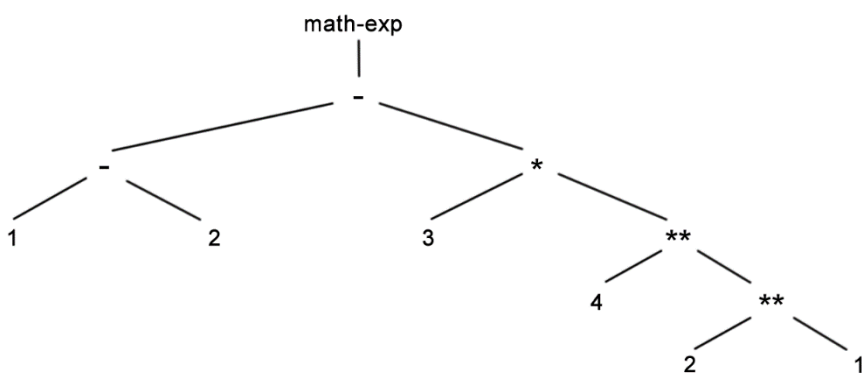
We start with the parse tree:



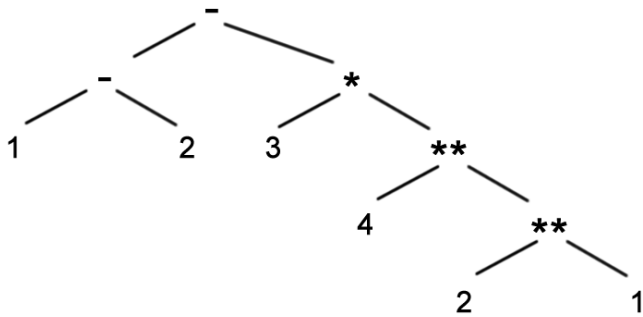
Now we remove the long chains of non-terminals and terminals with their matched text:



Then we bring the operators up to become subroots:



We can remove that math-exp at the root as well since it just forms a small chain with the non-terminal under it.

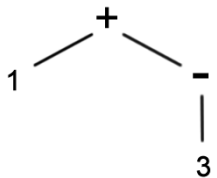


This is our finished abstract syntax tree.

What if we have a unary operator? For example, let's say that we support minus as either $1 - 2$ (a binary operator) or -3 (a unary operator). For unary operators, the operator is the parent node and the operand is the child node. For example, given the input:

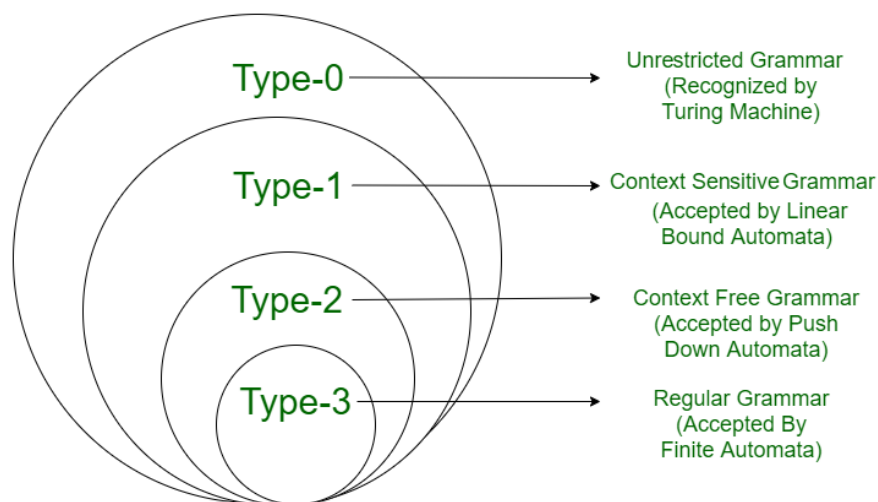
$1 + -3$

where $+$ is a binary operator and $-$ is used as a unary operator on the operand 3, the abstract syntax tree would be:



Regular Expressions

Most general purpose programming languages are "context free" and can be described using a context free grammar, such as the BNF format grammars above. A regular language, however, can be described using regular expressions and a DFA. This will be covered in more depth in CSC 310 Theory of Computing, but for now, this chart called the "Chomsky Hierarchy of Grammar" will help show the different levels of languages:



Regular expressions are not as powerful as a full programming language. There are no classes, functions, or loops. But they are plenty powerful enough for matching many different types of sequences. For example, let's say you have a language. First, what exactly is a language? A language is a set of all valid words and sentences. Well, what's a word? A word is a sequence of symbols. What's a symbol? A symbol is a single item in an "alphabet". What an alphabet? An alphabet contains the set of all symbols that you can use in that language.

Consider the following alphabet:

$$\Sigma = \{a, b\}$$

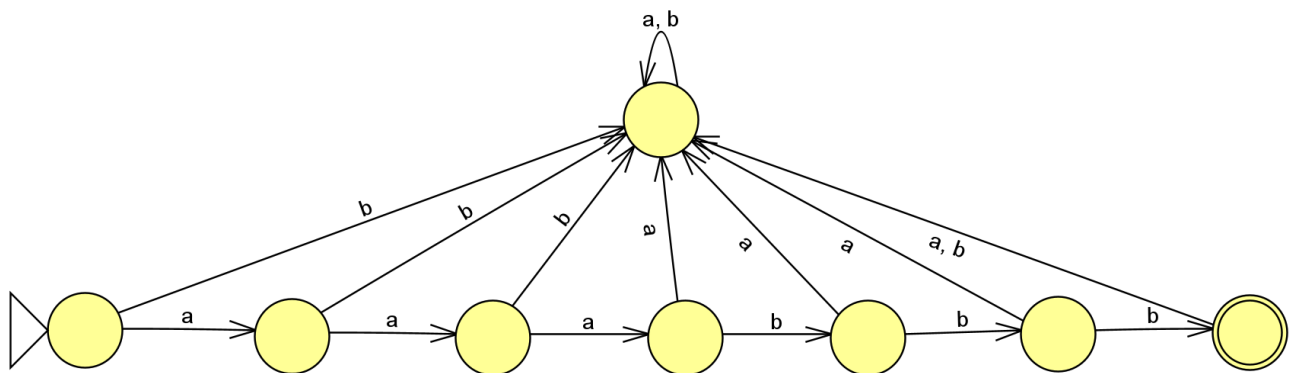
We use capital epsilon to denote an alphabet. This alphabet contains only the symbols "a" and "b". This means that all valid words in this language can only contains a's and b's as no other type of symbol is recognized. Let's say that a valid word or sentence (which is composed of 1 or more words) in our language is composed of 3 a's followed immediately by 3 b's. In other words:

$$L = a^3b^3$$

We use capital L to denote a language. We can represent this in regular expressions using the following pattern:

$$a\{3\}b\{3\}$$

This is a very simple example, as there is only one valid word in this language, namely "aaabbb". Since regular expressions are used to describe valid words in a regular language and can be matched using a DFA, the DFA for this simple language is the following:



We start in the left state. If we get an 'a', we move on. After three a's we are in the middle state. Three b's after that and we are in a final state. If we continue to get a's or b's, we move into an unrecoverable error state, where we stay indefinitely. If at any point we receive a symbol we weren't expecting (like a 'b' before we have seen exactly 3 a's) we move into the error state. A simple DFA, but it gets the job done.

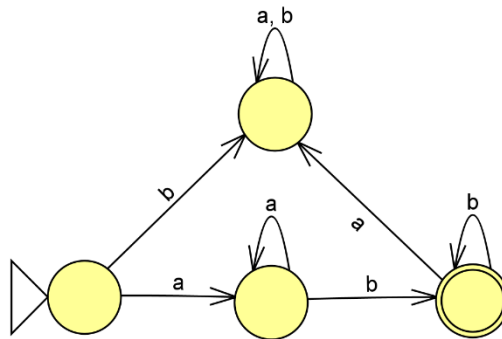
Now let's get a little more interesting. How about if we want 1 or more a's followed by 1 or more b's? In this case the exact count of a's and b's isn't important, as long as we have at least 1 'a' followed by at least 1 'b'. The language could be described as:

$$L = \{a^n b^m \text{ where } n \geq 1 \text{ and } m \geq 1\}$$

In regular expressions, this would be:

$$a^+b^+$$

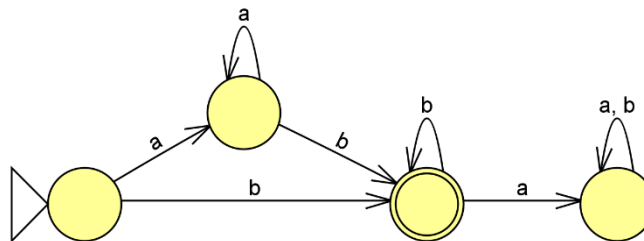
The plus operator matches 1 or more of the symbol that comes before it. Saying "1 or more" is equivalent to saying "at least 1". So, this literally says "at least 1 'a' followed by at least 1 'b'", which is exactly what we want. The DFA to recognize this language would be:



How about if a's are optional? If we want 0 or more a's we could do the following:

a^*b^+

The star operator allows for 0 or more of the symbol that comes before it. So, if we don't have any a's at all, that's ok. The DFA would be changed as follows:



Let's expand our alphabet to include all English letters, upper case and lower case. Let's say our language recognizes any word where capital letters come before lowercase letters. We can use the following regex for that:

$[A-Z]^*[a-z]^*$

Here we are using star, since I didn't say the word had to have capital or lower case, just that if it did, the capitals had to come first. I'm also using "character sets". Putting brackets around characters match a single character in that set. So $[A-Z]$ will match a single letter in the range of A through Z. Combined with the star that means 0 or more capital letters. Note that it does NOT have to be the same capital letter.

We could also match any word that ends with a vowel:

$.^*[aeiou]$

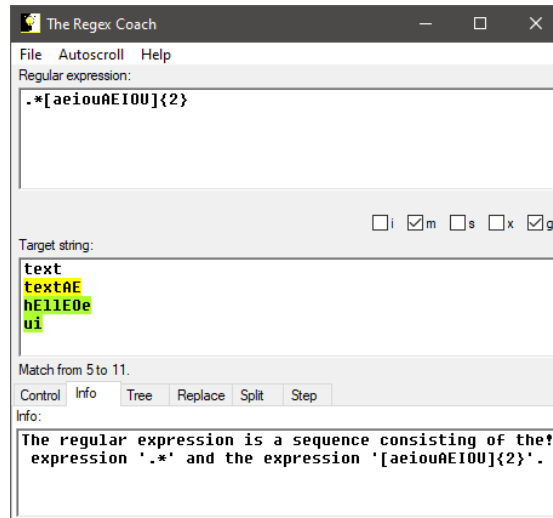
The dot operator matches any single character in our alphabet. Combined with the star, this matches any character any number of times. So literally anything! However, at the end we have used a character set and instead of using a range, we just enumerated all possible characters we are interested in. This means the word must end in either 'a', 'e', 'i', 'o', or 'u'. If we didn't care about case, we could rewrite it as:

$.^*[aeiouAEIOU]$

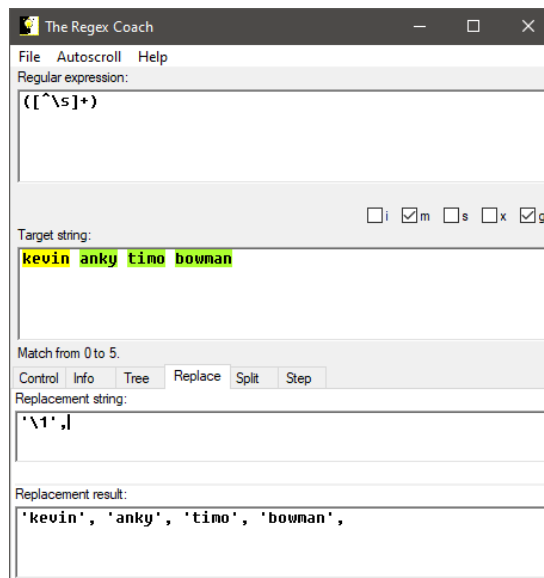
We could combine using the brace operator to get any word that ends in 2 vowels:

```
.*[aeiouAEIOU]{2}
```

If you want to play around with these, I suggest grabbing a tool such as [Regex Coach](http://weitz.de/regex-coach/)¹. I've been using this tool for years and can vouch to its usefulness. Try each of the regexs above out:



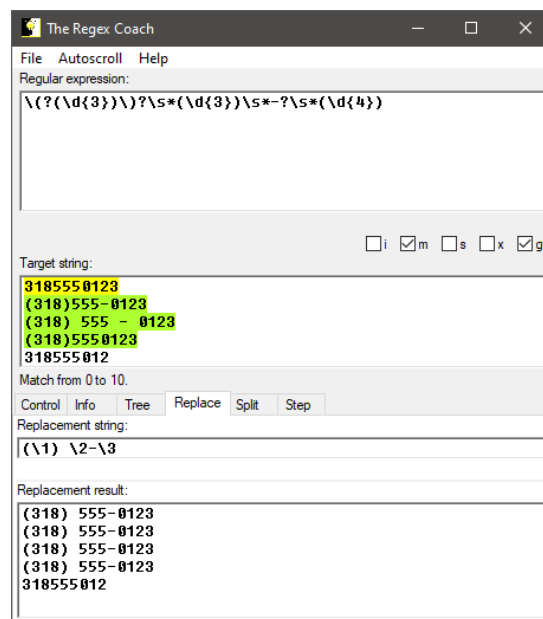
There are plenty more operators and things you can do with regex. It is a powerful tool for plenty of operations. For example, find and replace. Some editors will allow you to use regex to find and replace text in a document. Plenty of languages have regex built in so you can perform find and replaces programmatically in a large string. You can also use it for preprocessing large amounts of text. When you grab data from a source, rarely is it formatted the way you want it to be. You can use Regex Coach to preprocess that data far faster than you could do manually. As an example, imagine we have a bunch of names, and we want to put this into a list in Python. It is highly unlikely that the source you got the names from will have quotes around them and separate them with a comma, yet with a single regex we can fix that:



¹ <http://weitz.de/regex-coach/>

This uses some advanced regex features. Essentially it matches any character that is not a space character (the caret operator used in the beginning of the character set means to match everything BUT what's in the character set and \s stands for a space character, which is space, tab or newline). It matches 1 or more of these (using plus operator) and then "captures" that match (by putting parentheses around it). In the "Replace" section, I have used \1 to recall the first (and in this case only) match. If I had more matches (i.e. more sets of parentheses in the regex), \2 would be the second match, \3 would be the third, and so on. I surround that recalled match with single quotes and put a comma after it. Now all I have to do is copy and paste the final result into Python. All that's missing is an opening and closing bracket [] and I'll have to remove the last comma. Imagine if you had 50 names (or more)! Doing this by hand would be extremely time consuming. Regex is definitely worth learning!

We could also use regex for validation. For example, given a string, how would I know if it represents a valid phone number or not? A valid phone number has a 3 digit area code, then a 7 digit number. We could put the following regex into a language such as Javascript and programmatically check if the string is a valid phone number:



Notice how we not only verify it, but we can also replace it with our desired format for consistency. I'll leave it up to you to understand this regex, but to help you:

- `\ (` matches a literal left paren. Use backslash to match a literal character when that character has a different meaning in regex. In this case a left paren starts a matching group but if we want a literal left paren, we need to put a backslash before it.
- `?` makes the symbol (or group) before it optional. So, it might appear in the string, or it might not. This is different from `*` since with `?` it can only appear 0 or 1 times and not any more than that.
- `\d` matches any single digit from 0 to 9

We are going to focus on using regex for tokenization, but as you can see there are many more uses for it.

ANTLR

There are several tools out there to help you build an interpreter for your language. ANTLR² is one of them and it stands for ANOther Tool for Language Recognition.

ANTLR allows you to use regular expressions for your lexer rules so it can tokenize the input. It then matches those tokens to your production rules. The syntax is similar to BNF but there are some differences. ANTLR allows you to inject custom code (in the language of your choosing) to be executed when a production rule matches. In this way you can use it to build an interpreter. This isn't just about making your own interpreter for a programming language, you can use this to create your own DSL, or Domain Specific Language. This means you can create a language for use in a specific project. This doesn't have to be a programming language and it doesn't have to be a very complicated language. I have used ANTLR in an industry project to parse special input files. Ones that were formatted with a non-standard format, so no libraries existed to parse it. I have also used it in an open source project to allow execution of code while an application was running. In compiled languages, you can't usually take a string and interpret it as code while the app is running. This makes debugging hard since any changes to the code requires recompilation and executing the app again. I was able to use ANTLR to allow for custom code to be executed on-the-fly so, for example, if I'm making a game and I want to modify the properties of the final boss while the game is running, I could write code and execute it without stopping the game. It was very useful!

To use ANTLR, first you need to download the tools. The next programming assignment will talk more about this part. After you have the tools and libraries, you need to create a grammar file with a .g extension. This will contain everything ANTLR needs to create a parser for you. ANTLR works with many different languages, and you can tell it which language to generate code for. By default, it generates Java code. The first part of your grammar file is a name for your grammar, put after the "grammar" keyword:

```
// Grammar
grammar MyGrammar;
```

This will go at the top of your file. The double slash is just a single line comment and is obviously not required. After this you can set several options using the options keyword. You can set the programming language to generate code for, modify how tokens are generated, and more. You won't need this for the programming assignment, but you might want to look into what all you can do with this.

Next are the production rules. This is similar to BNF format but with a few changes. Assignment is colon and not ::= . Rules are ended with a semicolon. There is no epsilon; if you want to allow for empty string, you just leave it blank after the pipe |. There are no <> around terminals and non-terminals. And, as stated before, you can put custom code in with the rules, which means you can give rules a return value (like a function) and treat non-terminals/terminals like variables. With non-terminals you can pull out the value returned and with terminals you can grab the matched source text (e.g. if you have an identifier, you can grab the actual text of it, like "counter" or "foo"). Let's recall a grammar rule from before:

```
<func-mods> ::= <access-mod> <static> <final>
```

In ANTLR, this would be:

```
func_mods: access_mod static final;
```

² <https://www.antlr.org/download.html>

We would then have to define static, final, and access_mod:

```
access_mod: PUBLIC | PROTECTED | PRIVATE | ;  
static: STATIC | ;  
final: FINAL | ;
```

Notice the pipe right before the semicolon. This indicates that empty string is also acceptable. Terminal tokens are typically written in ALL_CAPS and we need to define them:

```
PUBLIC: 'public';  
PROTECTED: 'protected';  
PRIVATE: 'private';  
STATIC: 'static';  
FINAL: 'final';
```

Remember, you can think of terminals as constants, and they are essentially used as such in this context. Here is the full ANTLR .g file:

```
// Grammar  
grammar MyGrammar;  
  
// Parser  
func_mods: access_mod static final;  
  
access_mod: PUBLIC  
           | PROTECTED  
           | PRIVATE  
           | ; // empty  
  
static: STATIC  
       | ; // empty  
  
final: FINAL  
      | ; // empty  
  
// Lexer  
PUBLIC: 'public';  
PROTECTED: 'protected';  
PRIVATE: 'private';  
STATIC: 'static';  
FINAL: 'final';
```

Recall that we use regular expressions for our lexer rules. This can come in handy for certain terminal tokens. Remember the identifier rule?

```
<letter> ::= 'A'...'Z' | 'a'...'z'
<digit>  ::= '0'...'9'
<underscore> ::= '_'

<identifier> ::= <underscore> <identifier-rest>
               | <letter> <identifier-rest>

<identifier-rest> ::= <underscore> <identifier-rest>
                    | <letter> <identifier-rest>
                    | <number> <identifier-rest>
                    | ε
```

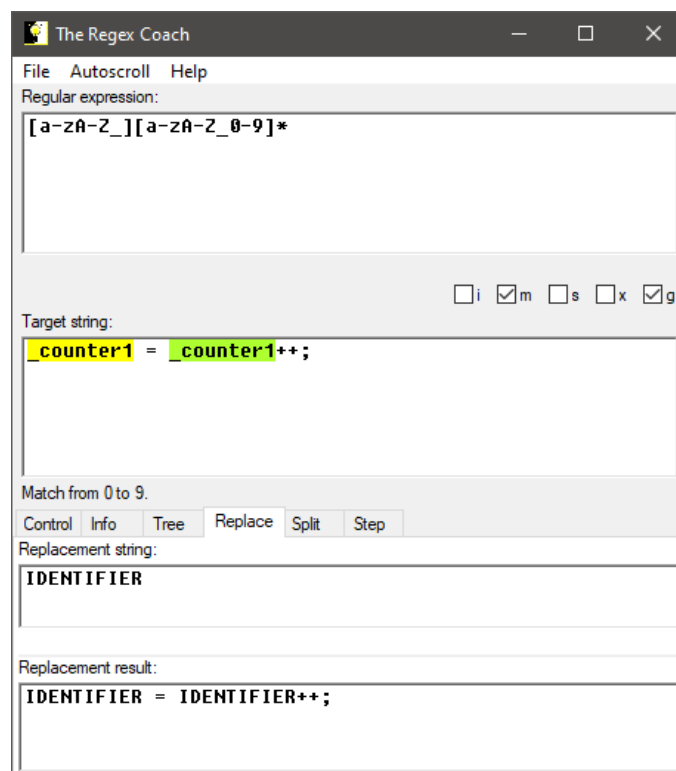
Identifiers really should be terminal tokens and we can consolidate this to show that. This is what the identifier rule would look like in ANTLR:

```
IDENTIFIER: [a-zA-Z_][a-zA-Z_0-9]*;
```

We have character ranges and operators to make this rule a lot nicer to read. If we really wanted to still have rules for letters, digits and underscore, we can use "fragments". Fragments are part of the lexer and aren't tokens themselves. Instead, they can be used as part of other lexer rules:

```
fragment DIGIT: [0-9];
fragment LETTER: [a-zA-Z];
fragment UNDER: '_';
IDENTIFIER: (LETTER | UNDER) (LETTER | UNDER | DIGIT)*;
```

Depending on your preference, this version might be easier to read. If you aren't sure about your regex, testing it in ANTLR can be a time consuming and tedious process. Remember you have Regex Coach:



We can turn our MyGrammar file into Java code by running the ANTLR tool on the command line:

```
$ antlr4 MyGrammar.g
```

The "\$" is just there to signify a terminal prompt. Several files will be generated when we run this. We will need to create some code to allow for an input to be given so we can give it to our lexer and parser. We then need to run the first parser rule. Parser rules become functions in the generated code. We can use this code to run our lexer and parser over input given from command line;

```
import org.antlr.v4.runtime.*;

public class Main {
    public static void main(String[] args) {
        String input = args[0].trim();
        System.out.println("\nInput is: '" + input + "'\n");
        var lexer = new MyGrammarLexer(CharStreams.fromString(input));
        var parser = new MyGrammarParser(new CommonTokenStream(lexer));
        parser.function_mods();
    }
}
```

This imports the ANTLR runtime. It then grabs input from command line arguments and turns this input into a character stream. It takes that character stream and gives it to the class MyGrammarLexer, which is one of the classes ANTLR generated from our grammar. That creates tokens, which we can turn into a token stream. That token stream is then fed into the class MyGrammarParser, which is another class generated by ANTLR. Finally, we run the function corresponding to our starting parser rule.

Now let's see what happens when we run this:

```
$ java Main "public static final"
```

```
Input is: 'public static final'
```

```
line 1:6 token recognition error at: ' ',
line 1:13 token recognition error at: ' ',
```

It seems to give us an error even when we give it perfectly fine code. The problem is that it doesn't know what to do with spaces. There is a space between static and final, then again between final and public. This may seem trivial, but the lexer has never seen this and has no rules on how to deal with it. So, it thinks it is a syntax error! We can fix this quite easily by adding the following at the end of our lexer rules:

```
WHITESPACE: [ \t\n\r\f]+ -> skip;
```

This creates a new token called WHITESPACE that matches either a space, a tab, or a newline. The "skip" part essentially tells the lexer to just skip over these tokens and don't actually put them into the token stream. So, all this does is tell the lexer to ignore all forms of whitespace. If we add this to our grammar, re-run the antlr4 tool, compile all of the .java files that were generated along with our own Main.java file, we will get this when it is executed:

```
$ java Main "public static final"
```

```
Input is: 'public static final'
```

No more syntax errors. Let's try a few more just to be sure:

```
$ java Main "private static final"
```

```
Input is: 'private static final'
```

```
$ java Main "static final"
```

```
Input is: 'static final'
```

```
$ java Main "final"
```

```
Input is: 'final'
```

```
$ java Main "protected"
```

```
Input is: 'protected'
```

What about when we give it incorrect syntax?

```
$ java Main "public static fin"
```

```
Input is: 'public static fin'
```

```
line 1:14 token recognition error at: 'fin '
```

```
$ java Main "fin"
```

```
Input is: 'fin'
```

```
line 1:0 token recognition error at: 'fin'
```

```
$ java Main "x"
```

```
Input is: 'x'
```

```
line 1:0 token recognition error at: 'x'
```

```
$ java Main "int counter"
```

```
Input is: 'int counter'
```

```
line 1:0 token recognition error at: 'i'
```

```
line 1:1 token recognition error at: 'n'
```

```
line 1:2 token recognition error at: 't'
```

```
line 1:4 token recognition error at: 'c'
```

```
line 1:5 token recognition error at: 'o'
```

```
line 1:6 token recognition error at: 'u'  
line 1:7 token recognition error at: 'n'  
line 1:8 token recognition error at: 't'  
line 1:9 token recognition error at: 'e'  
line 1:10 token recognition error at: 'r'
```

It seems to properly give us error messages (some more helpful than others). It was even able to recover from the error with "static fin public". It only gave us an error with "fin" and recovered with "public", not issuing an error for that part as well. So, this tool isn't too bad!

What about if we wanted to create an object that saved all this information. Perhaps it could have a boolean for isStatic and isFinal and could have a String for accessMod. It could look like this:

```
public class FunctionMods {  
    public String accessMod;  
    public boolean isStatic;  
    public boolean isFinal;  
  
    public FunctionMods() {  
        this.accessMod = "";  
        this.isStatic = false;  
        this.isFinal = false;  
    }  
  
    public FunctionMods(String accessMod, boolean isStatic, boolean isFinal) {  
        this.accessMod = accessMod;  
        this.isStatic = isStatic;  
        this.isFinal = isFinal;  
    }  
  
    @Override  
    public String toString() {  
        String am = accessMod;  
        if (am.equals("")) {  
            am = "none";  
        }  
        return "Access Mod? " + am +  
            "\nStatic? " + isStatic +  
            "\nFinal? " + isFinal;  
    }  
}
```

Can we get ANTLR to create an object of this class as it matches the appropriate keywords? Yes, and it isn't really that difficult.

As stated before, we can inject custom code into our parser rules. We do this by stating what the rule returns (remember parser rules are just functions in the generated code). We then initialize this returned value with the results of other non-terminal rules.

Here is what we can change our grammar file into:

```
// Grammar
grammar FuncMods;

// Parser
func_mods returns [FunctionMods value]
: access_mod s=static f=final {
    $value = new FunctionMods($access_mod.value, $s.value == 1, $f.value == 1);
}
;

access_mod returns [String value]
: PUBLIC {
    $value = $PUBLIC.text;
}
| PROTECTED {
    $value = $PROTECTED.text;
}
| PRIVATE {
    $value = $PRIVATE.text;
}
| /* empty */ {
    $value = "";
}
;

static returns [int value]
: STATIC {
    $value = 1;
}
| /* empty */ {
    $value = 0;
}
;

final returns [int value]
: FINAL {
    $value = 1;
}
| /* empty */ {
    $value = 0;
}
;

// Lexer
PUBLIC: 'public';
PROTECTED: 'protected';
PRIVATE: 'private';
STATIC: 'static';
FINAL: 'final';
WHITESPACE: [ \t\n\r\f]+ -> skip;
```

The parser rules now have the "returns" keyword added along with the datatype and variable name of what they return. It might seem odd to attach a variable name to the return type but ANTLR does this so you can simply assign a value to that variable and ANTLR will return that variable's value. You can grab the value from

other non-terminals as well. All variables have a dollar sign as a prefix. You can then use either a dot followed by the return variable, or a dot followed by "text". "Text" will retrieve the actual matched input. Note that you can also assign names to the non-terminals. I had to do this for "static" and "final" in the func_mods rule since those are keyword in Java. Using \$static.value in the code would result in "static.value" appearing in the generated code which would confuse the Java compiler.

Let's now change our Main class to the following:

```
import org.antlr.v4.runtime.*;

public class Main {
    public static void main(String[] args) {
        String input = args[0].trim();
        System.out.println("\nInput is: '" + input + "'\n");
        var lexer = new FuncModsLexer(CharStreams.fromString(input));
        var parser = new FuncModsParser(new CommonTokenStream(lexer));
        FunctionMods result = parser.function_mods().value;
        System.out.println("Result:\n" + result);
    }
}
```

After calling the func_mods method, we retrieve its value (the name of its returned variable). This is a FunctionMods object so we can grab this and print it out. Let's test this:

```
$ java Main "public static final"
```

```
Input is: 'public static final'
```

```
Result:
Access Mod? public
Static? true
Final? true
```

```
$ java Main "private static"
```

```
Input is: 'private static'
```

```
Result:
Access Mod? private
Static? true
Final? false
```

```
$ java Main "protected final"
```

```
Input is: 'protected final'
```

```
Result:
Access Mod? protected
Static? false
Final? true
```

```
$ java Main "static final"
```

```
Input is: 'static final'
```

```
Result:  
  Access Mod? none  
  Static? true  
  Final? true
```

Seems to be working pretty well. It even works for empty string, since that is technically valid:

```
$ java Main ""
```

```
Input is: ''
```

```
Result:  
  Access Mod? none  
  Static? false  
  Final? false
```

Usefulness

So, is all this knowledge only necessary if you get into compiler design? I mean, we can definitely agree that it will at least be useful knowledge to have if you do, but what else can you do with this?

Well, first off, as was mentioned above, regular expressions can be used in many areas. I personally have used reg ex in multiple contexts such as converting the format of student names and emails grabbed from Canvas (or Moodle) into a format I can paste into Excel. I've also used it to take a bunch of values and convert the format into proper list syntax for Python or C#. But aside from regular expressions, you can actually use parser tools such as ANTLR even when you aren't designing a compiler!

For example, let's say that you have an interesting custom format to represent data. We know that we can use JSON, XML or Binary to encode information, but these work best for serializing/deserializing into classes or dictionaries. If we don't want that, we could write our own format. We call this a "DSL" or "Domain Specific Language". If we know ANTLR and how to parse grammar, we can create our own syntax for our DSL and write a parser to decode it. In fact, I personally have done this for an industry project. The DSL format was already figured out when they hired me and part of my job was to write code to parse a DSL file, allow editing of the contents, then write the modified information back into the file. This required parsing tools and knowledge of how this all works. Honestly, with the proper knowledge and using tools like ANTLR, I found this part of the project to be the easiest (and the most fun). I had complete control over the parsing, and I was even able to make changes when the DSL format changed. So even though this had nothing to do with writing a compiler, and I wasn't parsing an actual programming language, this knowledge was still hugely beneficial, and all the same techniques still applied. So, this grammar stuff isn't just theoretical nor is it only important if you go into compiler design.

Conclusion

There is a lot to this lesson and actually I'm just scratching the surface! This is the beginning of what it takes to create an interpreter, and a full on compiler has many more steps. Hopefully now you get an idea of what goes on when you call a program such as "javac" on the command line or run "python" with a given file. Fortunately, if you understand the basics, tools like ANTLR can help you create your interpreter. As long as the grammar is simple, it isn't too bad overall. Your next programming assignment is exactly this. You will be creating your own interpreter for the grammar from homework 1.